VIT Bhopal University

VITyarthi - Build Your Own Project

The Alcove

A Small Library Management System in Python

Project Report

Course	CSE1021 - Introduction to Problem Solving and Programming
Student	Aditi Singh
Register No.	26BCE11020
Programming language	Python 3 (standard library only)
Submitted	30 September 2026

Contents

2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots and Results
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. References

2. Introduction

The Alcove is a menu-driven library manager written in pure Python using only the standard library. It lets a librarian manage a catalogue of books and a list of members, issue and return books, charge late fines automatically, and view simple reports such as the most popular books and the list of overdue loans. All data is kept in a single JSON file, so the program needs no database, no internet connection and no installation beyond Python itself.

The project was built for the course CSE1021 (Introduction to Problem Solving and Programming) and applies its core topics: functions and modules, lists, dictionaries and tuples, searching and sorting, a small numeric algorithm (the ISBN-13 check digit), file handling and exception handling.

3. Problem Statement

Small community libraries - a hostel reading corner, a club shelf, a cafe with a bookcase - are usually run from a paper register or a spreadsheet. This causes familiar problems: typing mistakes in ISBNs, fines worked out by hand, and no idea which books are actually popular. Full library software is too heavy for such places.

The Alcove is a small, menu-driven library manager that checks its own data, enforces simple borrowing rules and reports on the borrowing history.

Objectives

- Keep an accurate catalogue of books, rejecting mistyped ISBNs using the ISBN-13 check digit.
- Register members with short, readable IDs (M001, M002, ...).
- Enforce the house rules automatically: 14-day loans, at most 3 books per member, Rs. 2 fine per late day, fines paid before new loans.
- Provide reports that help the librarian: summary, most popular books, overdue books, books per genre.
- Never crash on bad input; save data automatically; cover the logic with automated tests.

Scope and target users

In scope: book and member management, issue/return with fines, four reports, JSON persistence and unit tests. Out of scope: graphical or web interface, multi-user access, real payments, barcode scanning, notifications, waitlists and renewals. Target users are the librarian of a small library and first-year programming students who can read the code as a worked example.

4. Functional Requirements

The system has four major functional modules with a clear input/output structure: the user picks a numbered menu option, types validated inputs, and the program prints a message or a table.

ID	Module	Requirement	Input -> Output
FR1	Books	Add a book with ISBN-13 validation, title, author, optional genre and 1-50 copies	ISBN, text, number -> 'Book added.'
FR2	Books	View all books sorted by title, with copies in/total	menu choice -> table
FR3	Books	Search by keyword in title, author or genre	keyword -> matching rows
FR4	Books	Update title/author/genre (blank keeps old value); remove a book only if no copy is on loan	ISBN + fields -> confirmation
FR5	Members	Add a member (ID auto-generated), list members with fines due, view a member's loans, remove a member with no loans or fines	name / ID -> ID, table
FR6	Counter	Issue a book: due date = issue date + 14 days; refuse if fines unpaid, 3 loans held, duplicate copy, or no copies left	member ID, ISBN -> due date or error
FR7	Counter	Return a book; fine = Rs. 2 x days late is added to the member's account	member ID, ISBN -> fine amount
FR8	Counter	Pay a member's outstanding fines	member ID -> amount paid
FR9	Reports	Library summary; top-K most borrowed books; overdue books with fine so far; books per genre	menu choice (K) -> table
FR10	System	Save all data to a JSON file after each action; load it on start; add sample data on first run; change 'today' for demos	-

5. Non-functional Requirements

Requirement	How The Alcove meets it
Reliability / error handling	All library-rule violations raise one exception type, LibraryError, caught in the main loop and shown as a friendly '!' message. A missing, unreadable or wrongly structured data file starts an empty library instead of crashing.
Usability	Numbered menus, prompts that repeat until the input is valid (read_int, read_isbn, read_text, confirm), aligned text tables, IDs that are short and readable.
Maintainability	Small modules with one job each; house rules are named constants (LOAN_DAYS, MAX_LOANS, FINE_PER_DAY) changed in one place; docstrings and comments throughout.
Portability	Python 3.8+ standard library only; runs the same on Windows, macOS and Linux; no installation step.
Data integrity	Books on loan cannot be removed; members with loans or fines cannot be removed; duplicate ISBNs are rejected; member IDs are never reused.
Performance / efficiency	Books and members are stored in dictionaries, so look-ups by ISBN or ID take constant time; the data set of a small library fits easily in memory.
Testability	Business logic is separate from input/output, so it is tested directly; the menu layer is tested by feeding scripted input.
Security (limitations)	The program runs locally with no network access. User authentication and encryption of the data file are out of scope and listed as future work.

6. System Architecture

The program has two layers. The presentation layer (main.py) only talks to the user: it prints menus, reads input and prints results. The business-logic package (alcove/) holds the real work, one module per concern. All data lives in one dictionary called state, which storage.py saves to and loads from data/alcove_data.json.

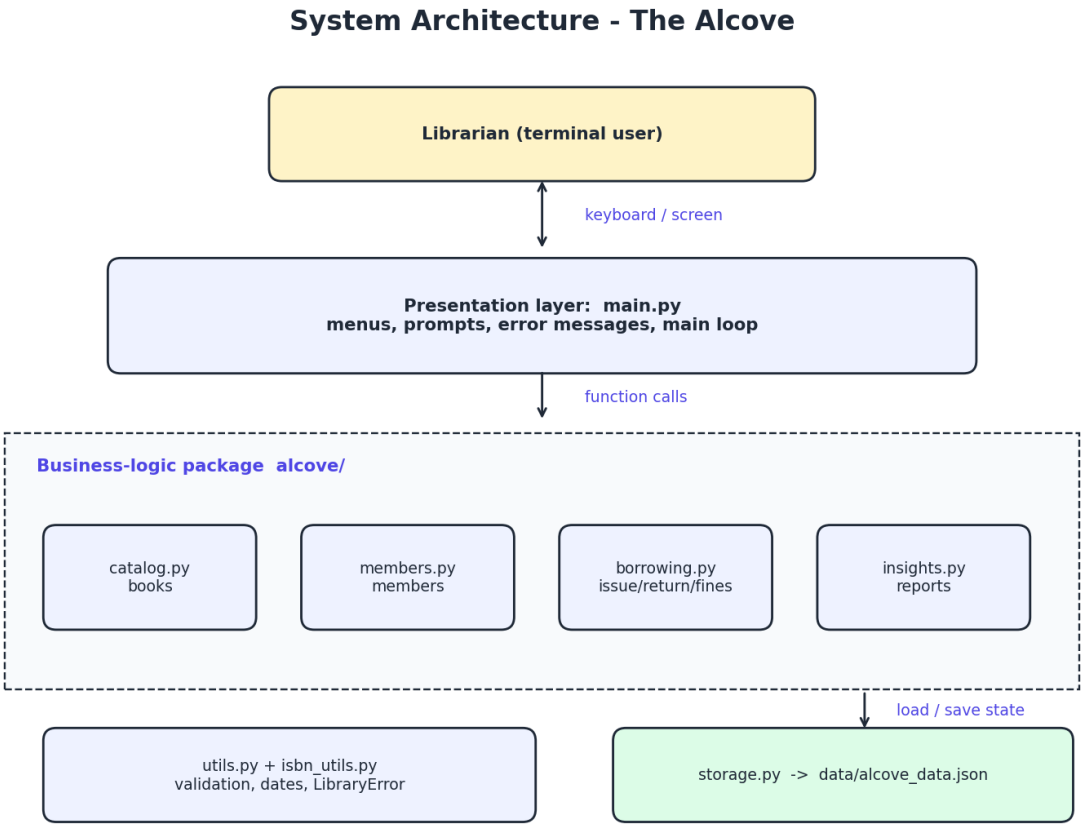


Figure 1 - System architecture

Module	Responsibility
main.py	Menus, prompts, main loop, sample data, error display
alcove/catalog.py	Add, find, update, remove and search books
alcove/members.py	Add, find, list and remove members; ID generation
alcove/borrowing.py	Issue, return, fines, house-rule constants
alcove/insights.py	Summary, top-K books, overdue report, genre counts
alcove/isbn_utils.py	ISBN-13 clean-up and check-digit validation
alcove/utils.py	Input helpers, date helpers, LibraryError
alcove/storage.py	Load and save the JSON data file

7. Design Diagrams

7.1 Use case diagram

The librarian is the only actor. The use cases correspond to the menu options of the program.

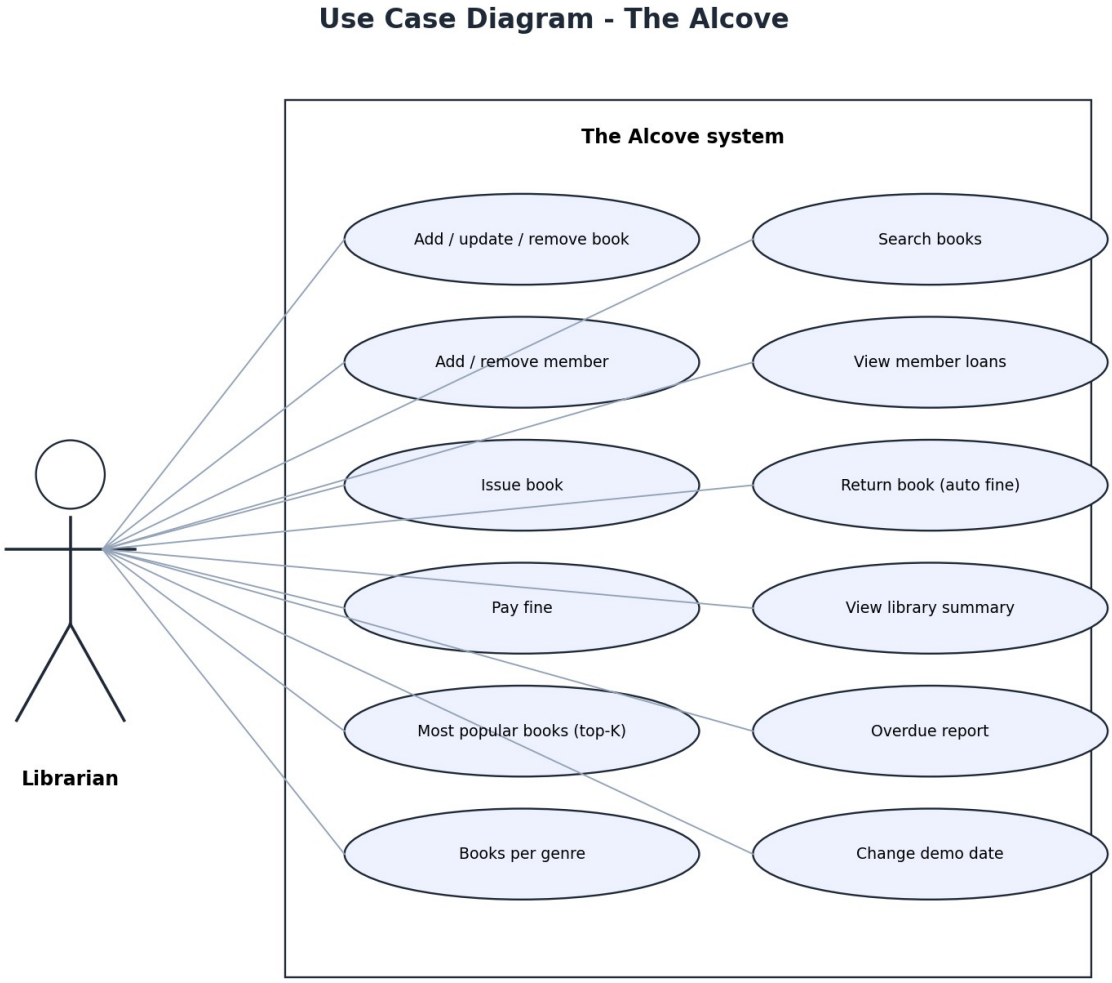


Figure 2 - Use case diagram

7.2 Workflow diagram

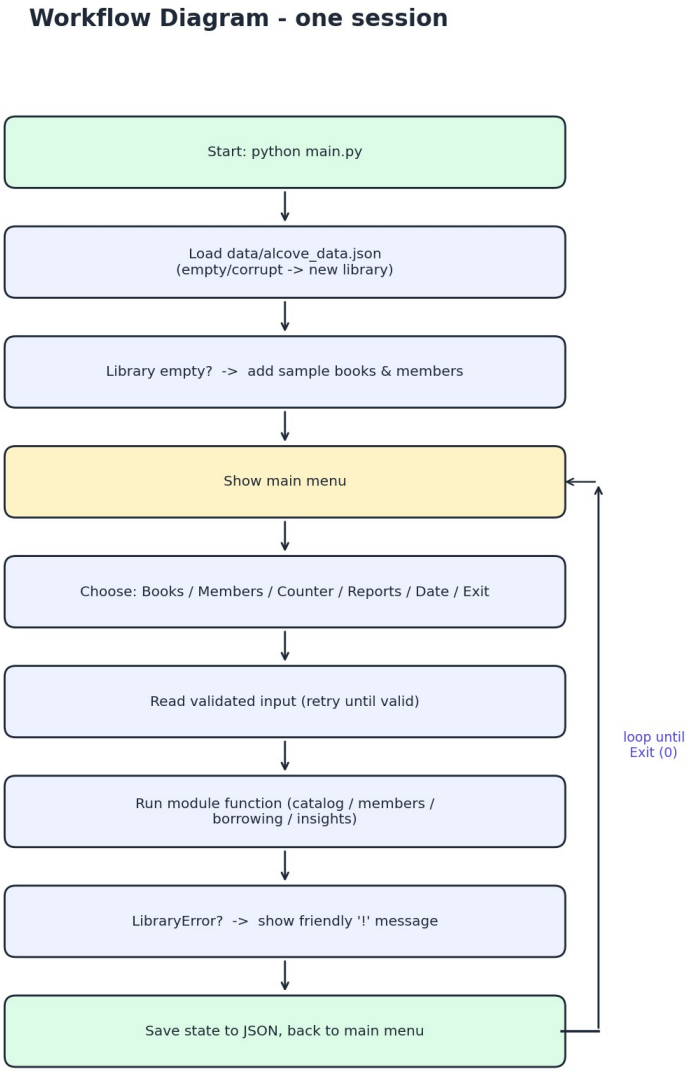


Figure 3 - Workflow of one session

7.3 Sequence diagram

Issuing a book is the most rule-heavy operation, so it is shown as the sequence diagram.

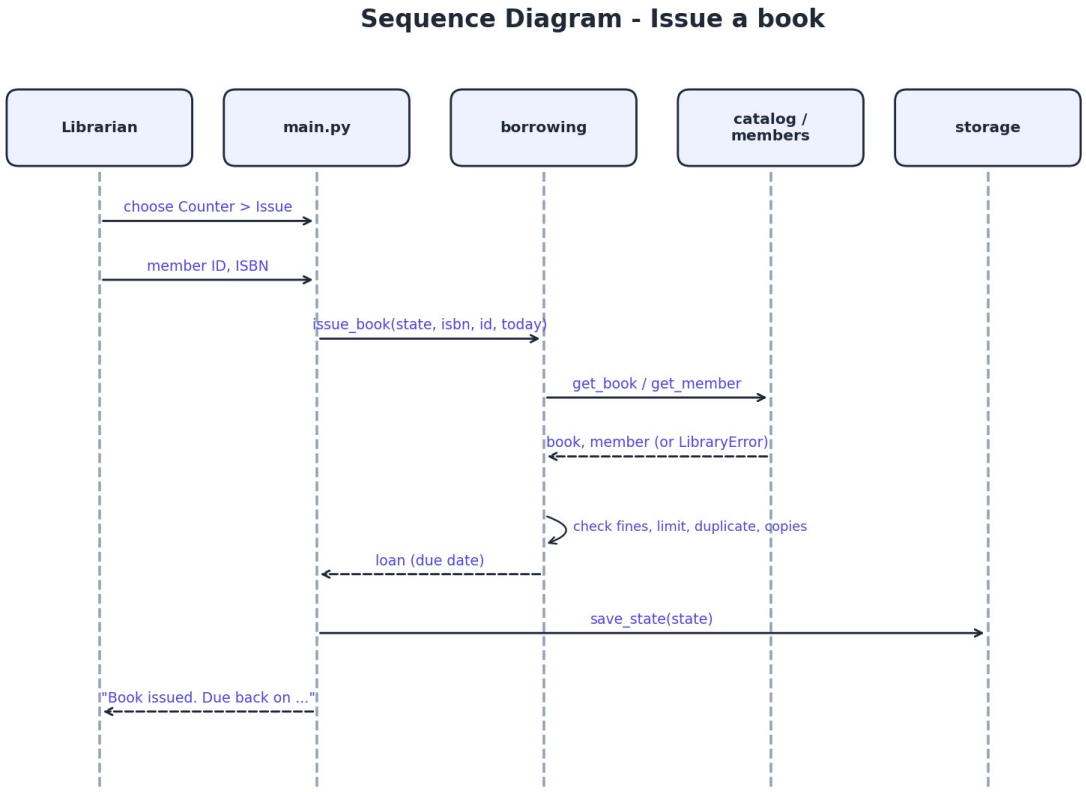


Figure 4 - Sequence diagram: issue a book

7.4 Class / component diagram

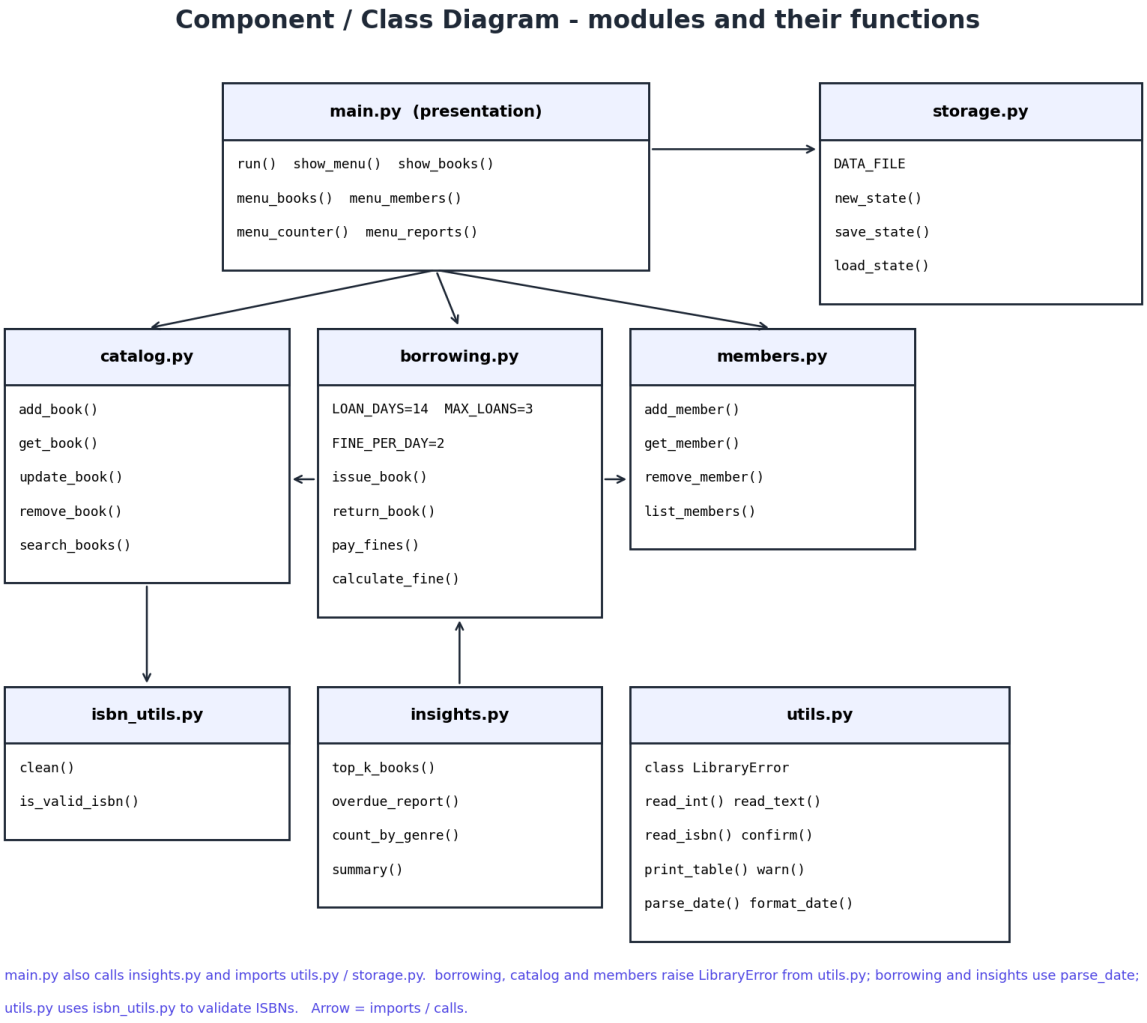


Figure 5 - Component diagram: modules and functions

7.5 ER diagram and storage design

The project uses a JSON file instead of a database, but the data still has a clear structure with three entities. In the file, books and members are dictionaries keyed by ISBN and member ID, loans are a list, and next_member_id is a counter.

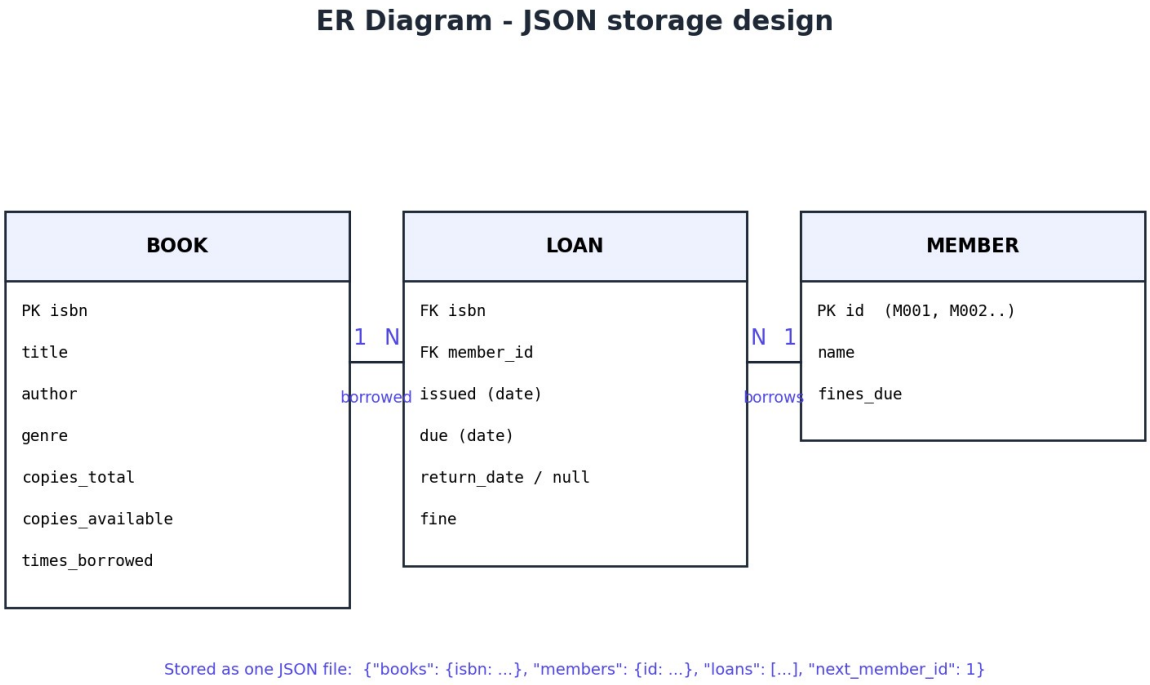


Figure 6 - ER diagram of the stored data

Entity	Fields
Book	isbn (key), title, author, genre, copies_total, copies_available, times_borrowed
Member	id (key, e.g. M001), name, fines_due
Loan	isbn, member_id, issued, due (dates as YYYY-MM-DD text), return_date (null while on loan), fine

8. Design Decisions and Rationale

- **Standard library only.** No installation or internet is needed, which suits a small library and makes the project easy to run and mark.
- **JSON file instead of a database.** The data set is small; JSON is human-readable, easy to back up and needs no server. The cost is that the whole file is rewritten on every save, which is fine at this scale.
- **Single state dictionary.** Every function receives the state and changes it, so functions are easy to test with a small hand-built state and no file access.
- **Separate user interface from logic.** main.py never applies rules; modules never call print or input for library rules. This is why the logic can be unit tested directly.
- **One exception type.** LibraryError signals every broken rule; the main loop catches it in one place, so no operation can crash the program with a rule violation.
- **ISBN-13 check digit.** It catches most typing errors at the moment of entry, addressing the paper-register problem directly.
- **Rules as constants.** LOAN_DAYS, MAX_LOANS and FINE_PER_DAY sit at the top of borrowing.py so a library can change its policy in one line.
- **Change-date option.** Fines depend on dates, so a demo option lets the user move 'today' forward and see fines without waiting.
- **Fines block new loans.** Unpaid fines must be cleared before a member can borrow again; this keeps the fine rule meaningful.

9. Implementation Details

The project has 8 source files (main.py plus 7 modules) and 8 test files. Key algorithms and data structures are described below.

ISBN-13 validation (isbn_utils.py)

Digits are weighted 1, 3, 1, 3, … and the total including the check digit must be divisible by 10. Input with non-ASCII digits is rejected so that look-alike characters cannot slip through.

```
def is_valid_isbn(text):
    code = clean(text)
    if len(code) != 13 or not (code.isascii() and code.isdigit()):
        return False
    total = 0
    for position in range(13):
        weight = 1 if position % 2 == 0 else 3
        total += int(code[position]) * weight
    return total % 10 == 0
```

Issuing a book (borrowing.py)

issue_book looks up the book and member, then checks four rules in order: unpaid fines, the 3-book limit, a duplicate copy already held by the member, and copies available. Only when all pass does it create the loan record, reduce copies_available and increase times_borrowed, so a failed attempt never leaves half-changed data.

Fine calculation

```
def calculate_fine(due_text, returned_on):
    days_late = (returned_on - parse_date(due_text)).days
    if days_late <= 0:
        return 0
    return days_late * FINE_PER_DAY
```

Reports (insights.py)

- Top-K books: sorts the books by times_borrowed (descending) and takes the first K.
- Overdue report: scans loans not yet returned and computes days late and fine so far as of 'today'.
- Books per genre: counts titles with a dictionary (genre -> count).
- Summary: adds up copies, copies on loan, members and outstanding fines.

Input handling and storage

The read_int, read_text, read_isbn and confirm functions loop until the user gives valid input, so the rest of the program never sees bad values. save_state writes the dictionary as indented JSON, creating the data folder if needed. load_state returns a new empty library if the file is missing, unreadable or has an unexpected structure, and prints a warning.

10. Screenshots and Results

All screens below show the running program; the sample data created on first run is used. The dates are the ones the program printed when the screenshots were taken.

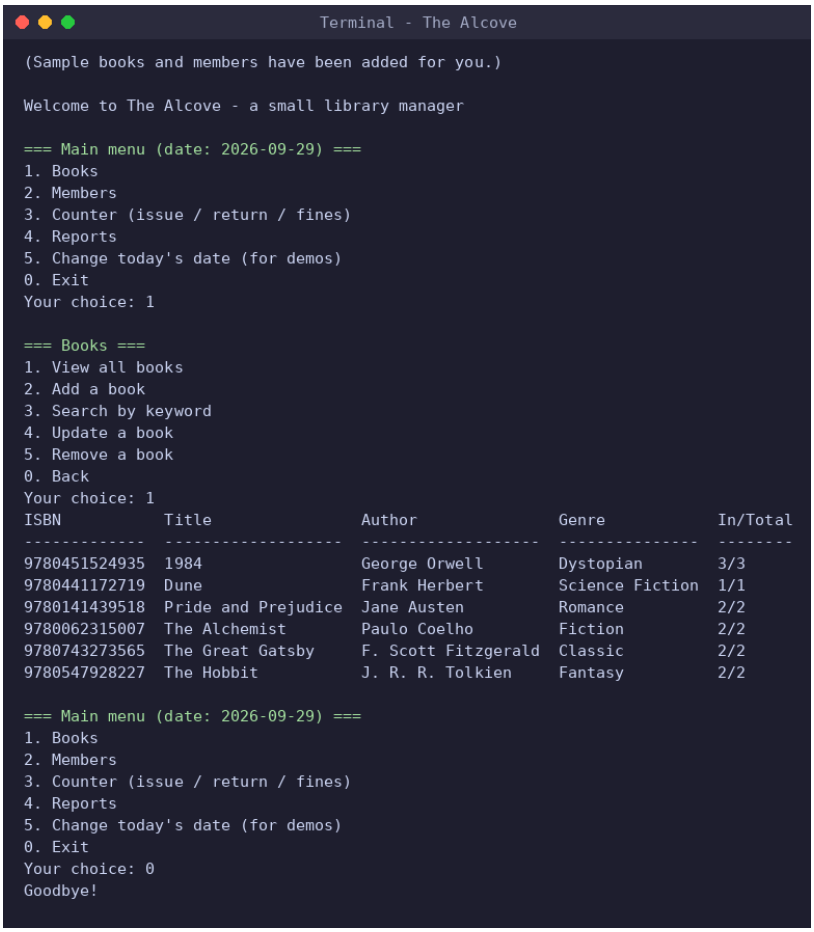


Figure 7 - Main menu, Books menu and the list of books

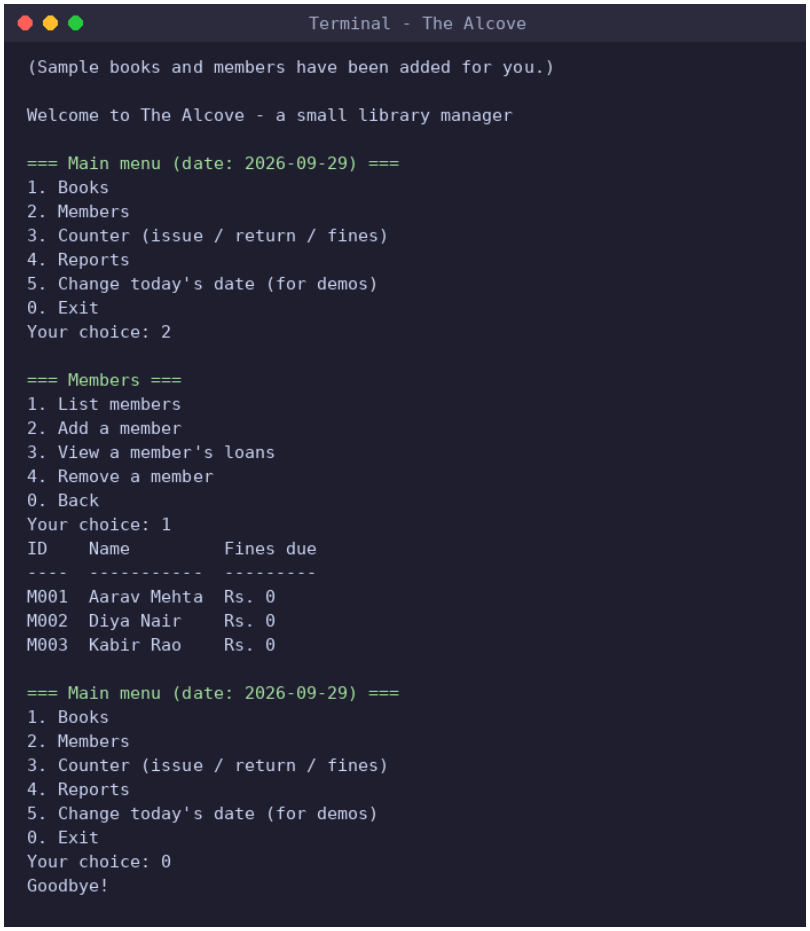


Figure 8 - Listing members



Figure 9 - Keyword search (fantasy)



Figure 10 - Issuing 'Dune' to member M001 (due in 14 days)

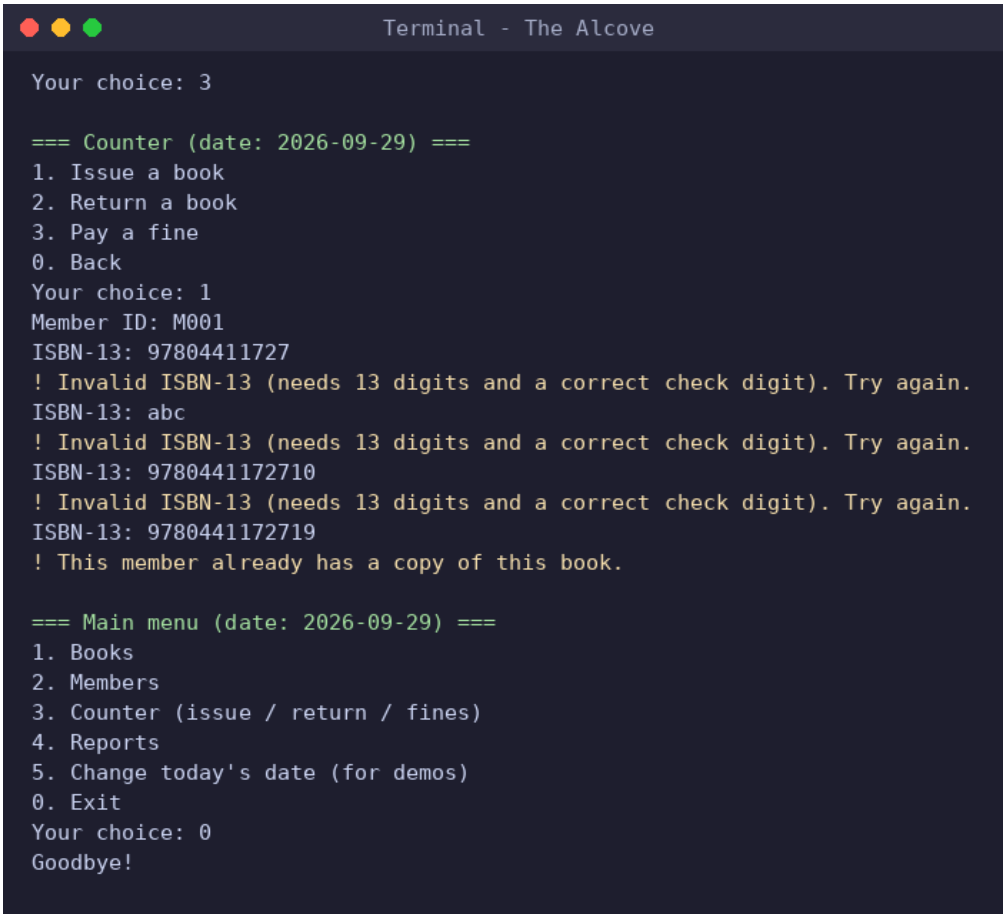


Figure 11 - Error handling: no copies left, invalid ISBNs re-asked, duplicate loan refused

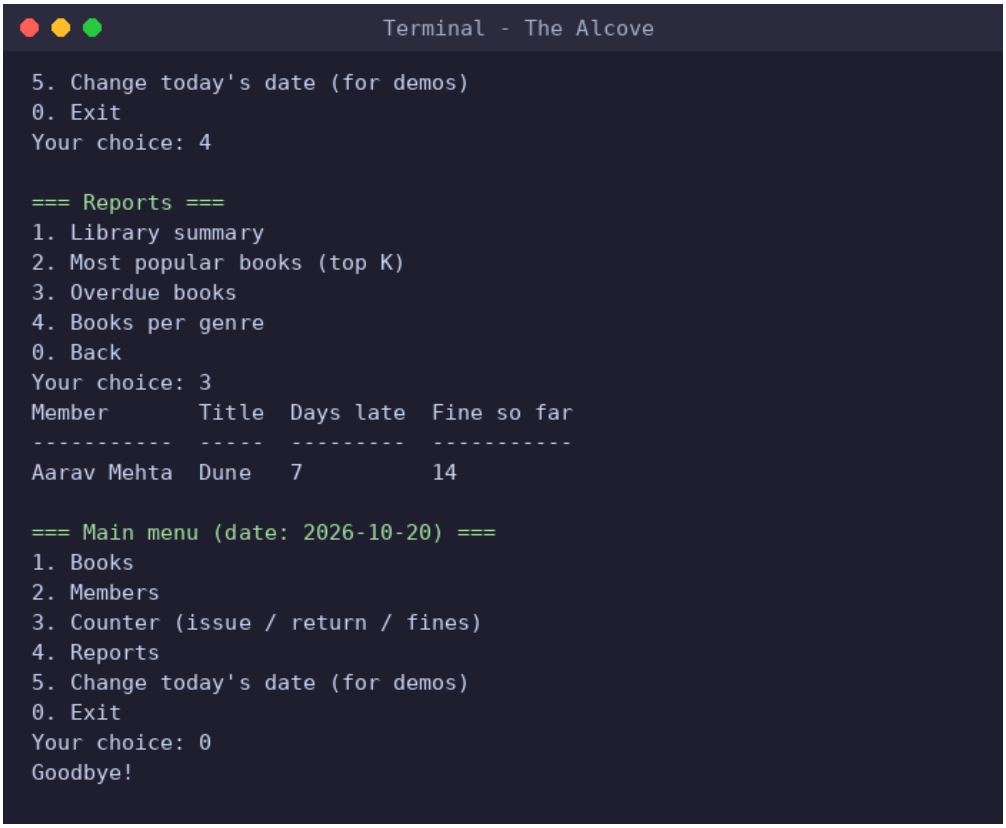


Figure 12 - Overdue report after moving today's date 21 days ahead (7 days late, fine Rs. 14)

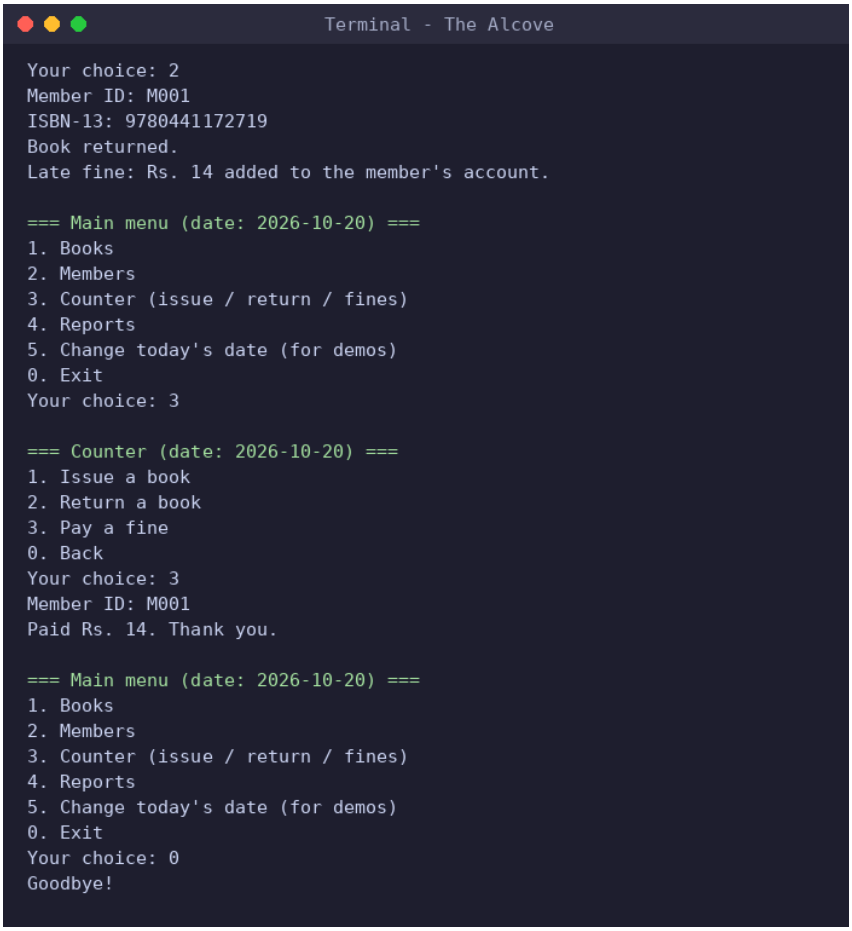


Figure 13 - Late return adds a fine; the fine is then paid

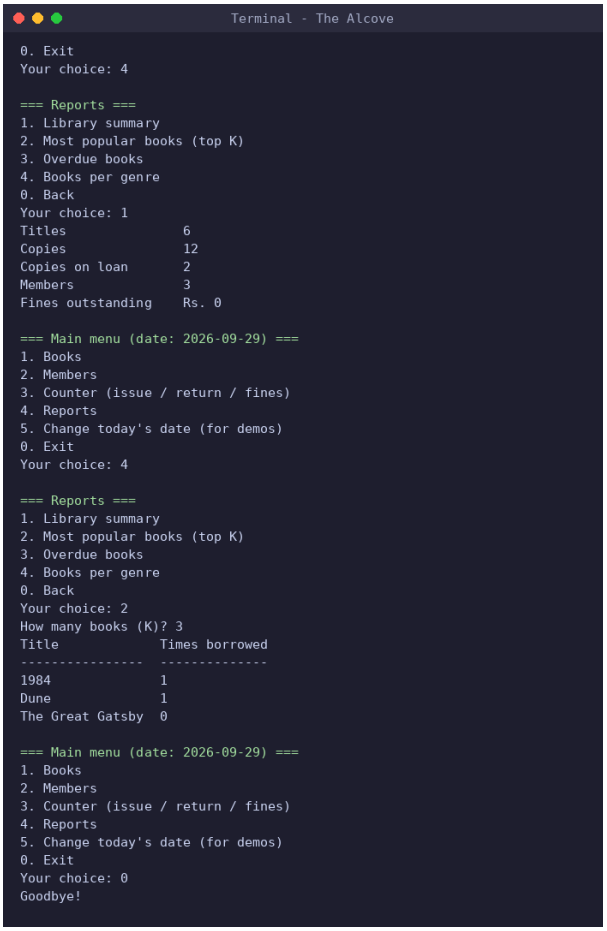


Figure 14 - Library summary and top-3 most popular books

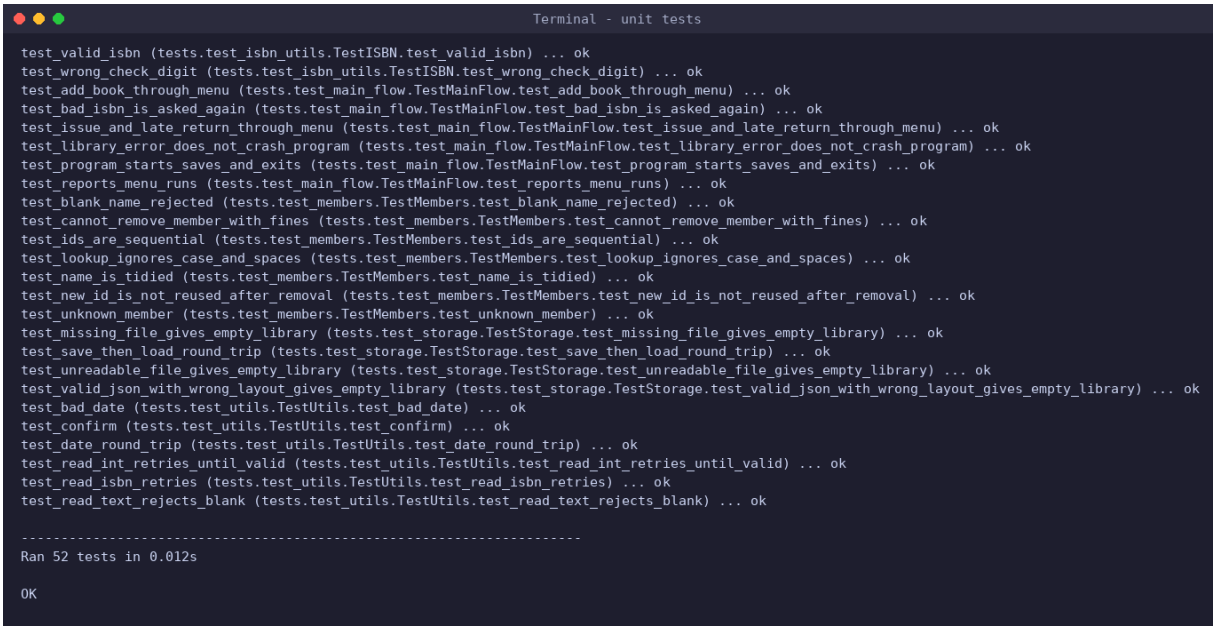


Figure 15 - All 52 automated tests passing

11. Testing Approach

The project has 52 automated tests written with Python's unittest module, run with: python -m unittest discover -s tests -t . -v

Test file	What it checks
test_isbn_utils.py	Valid and invalid ISBNs, hyphens, wrong length, letters, non-ASCII digits
test_catalog.py	Adding, duplicate ISBN, blank fields, update, remove rules, search
test_members.py	ID generation, IDs not reused, name tidying, removal rules, unknown member
test_borrowing.py	Issue rules (fines, limit, duplicate, no copies), return, fine amounts, paying fines
test_insights.py	Top-K ordering, overdue rows, genre counts, summary numbers
test_storage.py	Save/load round trip, missing file, unreadable file, wrong layout
test_utils.py	Input retry loops, date parsing, yes/no prompts
test_main_flow.py	End-to-end runs through the menus with scripted input, including a rule error that must not crash the program

Result: all 52 tests pass (see Figure 15). Tests use small hand-built states and temporary files, so they never touch the real data file.

12. Challenges Faced

- Keeping the user interface and the rules apart, so the rules could be tested without typing into a terminal.
- Deciding the exact rules for fines (per-day rate, when they are charged, and that they block new loans) and making them consistent across issue, return, payment and reports.
- Making a failed operation leave data unchanged - all checks in issue_book run before any change is made.
- Handling bad input everywhere without crashing: input loops for typing errors, one exception type for rule errors, and safe recovery from a damaged data file.
- Testing date-dependent behaviour; this led to passing the date into functions instead of reading the clock inside them.
- During a final review, some edge cases were found and fixed: look-alike Unicode digits in an ISBN, a JSON file that is valid but has the wrong structure, and Ctrl+C at a prompt.

13. Learnings and Key Takeaways

- Splitting a program into small modules makes it easier to write, test and explain.
- Passing data in as arguments (the state dictionary, today's date) makes functions predictable and testable.
- Validating input at the boundary keeps the rest of the code simple.
- Automated tests catch mistakes quickly when rules change, and writing them exposes edge cases.
- Version control and clear documentation (README, problem statement, diagrams) are part of the work, not an afterthought.

14. Future Enhancements

- A graphical or web interface (for example Tkinter or Flask).
- Librarian login and encryption of the data file.
- Renewals, reservations and waitlists for popular books.
- Barcode scanning of ISBNs and e-mail or SMS reminders for due dates.
- Switching from JSON to SQLite for larger libraries and safer concurrent access.
- Exporting reports to CSV or PDF.

15. References

1. Python Software Foundation. Python 3 documentation - json, datetime, unittest. https://docs.python.org/3/
2. GS1. How to calculate a check digit manually (ISBN-13 uses the same EAN-13 method). https://www.gs1.org/services/how-calculate-check-digit-manually
3. International ISBN Agency. ISBN Users' Manual. https://www.isbn-international.org/
4. Course material: CSE1021 - Introduction to Problem Solving and Programming, VIT.
5. VITyarthi. Build Your Own Project - General Project Instructions and Submission Guidelines.